C++ 大作业

项目简介

名称：Not necessarily an STG “并非 STG”

制作：李梓萌 2024303504

项目架构

综述

本项目核心围绕现代的实体-组件-系统（ECS）架构构建，使用了 EnTT 库来实现相对纯粹的 ECS 模型。

项目采用以 `Game` 类为核心的 **单例架构**，通过 EnTT 的上下文系统统一管理所有子系统。初始化过程遵循“先初始化核心系统，再初始化游戏对象”的顺序，确保依赖关系清晰有序。

在设计理念方面，本项目有几个特点：

1. **ECS 架构为基础**：项目使用 EnTT 提供的 ECS 模型，实体只是轻量的标识符，组件只承载数据而不包含行为，而系统则操作特定组件组合。通过 `game` 命名空间提供了全局访问的工具函数。
2. **基于上下文的子系统管理**：所有核心子系统都存储在注册器上下文中，并通过统一的 Getter 方法访问，包括：
   - **Logger**：基于工作线程的异步日志系统；
   - **ThreadPool**：多线程任务调度执行；
   - **ResourceManager**：带有缓存机制的资源加载系统；
   - **Window**：具备后处理效果的渲染系统。
3. **层级场景树结构**：通过组件实现父子关系的场景树，实体可以挂载在场景树上，并相对于父节点进行定位，实现游戏对象的层次结构。
4. **事件驱动的系统通信**：项目利用 EnTT 的事件调度器实现系统间的解耦通信，使游戏逻辑与 UI 系统之间保持清晰分离。

5. **分层渲染架构**：渲染系统支持多个渲染目标和图层排序，同时具备像素化、CRT 效果和泛光等后处理功能。

此外，该项目还体现了以下基础设施设计模式：

- 使用工具函数实现全局访问抽象；
- 懒加载结合缓存的资源管理机制；
- 异步处理提升响应与性能；
- 组件组合优于传统继承方式；
- 通过预制件（Prefab）机制复用游戏对象模板。

核心架构

实体-组件-系统（ECS）基础

该项目采用 **EnTT 库** 实现完整的 ECS 架构，核心围绕以下三大概念展开：

- **实体（Entity）**：由中心注册器统一管理的唯一标识符
- **组件（Component）**：用于描述实体属性的数据容器
- **系统（System）**：处理特定组件组合的逻辑单元

单例式游戏管理器

项目通过 **单例模式** 构建了主 `Game` 类，作为所有子系统的统一协调者。`Game` 类提供了对以下内容的访问接口：

- 实体注册器和事件调度器
- 资源管理系统
- 多线程与日志系统
- 窗口与输入管理

基础设施组件

资源管理系统

项目基于 EnTT 的资源缓存功能实现了高级的 **资源缓存机制**，`ResourceManager` 提供类型安全的缓存支持，适用于以下资源：

- 纹理与精灵帧
- 字体与文本资源
- 二进制文件与通用资产
- 着色器与视觉特效
- 剧情对话数据等叙事资源集合

多线程基础设施

项目内建了 **线程池支持**，用于异步操作处理。其核心特性包括：

- 基于硬件并发能力可配置的线程数量
- 支持任务执行及完成回调
- 支持同步与异步执行模式

日志系统

项目实现了一个 **线程安全的日志系统**，支持多个日志等级。日志器运行在独立线程中，避免阻塞主线程游戏逻辑，并支持从调试级别到致命错误的不同详细程度。

设计模式与架构理念

预制件（Prefab）实体构建模式

项目采用 **预制件模式** 来创建复杂实体，通过静态工厂方法进行构造。所有预制件类均继承自一个通用的 `TreeLike` 基类，该类负责 RAII 风格的实体生命周期管理。

层级场景管理系统

项目实现了一个 **场景树系统**，允许实体之间形成父子层级关系，具备以下特性：

- 相对位置与变换
- 父实体销毁时自动清理子实体
- 高效的变更传播（基于脏标记机制）

组件组合式架构

项目坚持 **组合优于继承** 的设计理念，组件划分为不同功能类别：

- **变换组件（Transform）**：用于位置、缩放等空间信息
- **渲染组件（Render）**：用于视觉表现
- **游戏逻辑组件**：处理特定实体行为
- **场景树组件**：处理层级关系与结构

事件驱动架构

项目使用 **事件调度系统** 实现子系统间的解耦通信，事件用于处理游戏状态变更、实体交互等逻辑，提升系统模块化程度。

系统组织结构

模块化系统设计

项目将功能划分为多个独立模块系统，每个系统专注于特定组件组合的处理，例如：

- RenderControl：处理渲染逻辑
- MovementControl：实体移动控制
- CollisionControl：物理碰撞与检测
- SceneControl：场景结构管理
- MusicControl：音频播放管理

全局访问模式

项目提供了统一接口形式的 **全局访问函数**，可用于访问核心系统，并包括常用工具方法，例如随机数生成、实体查询等。

技术栈

该项目基于现代 C++ 构建，选用了若干高质量库组件：

- **SFML**：负责图形、音频与窗口管理
- **EnTT**：提供 ECS 架构与资源管理支持
- **nlohmann::json**：json 解析
- 使用 **现代 C++ 特性**，诸如高级模板元编程与 RAII 自动资源管理模式

渲染管线架构

本项目实现了一套基于 SFML 的多通道渲染架构，在主循环函数 `Window::run()` 中组织整个渲染流程。渲染目标被明确分离为多个 RenderTexture，用于绘制不同类型的内容，包括主游戏内容（如玩家、敌人、子弹等）、用户界面元素、光照效果，以及小地图。这些渲染目标通过 `CRenderTargetComponent` 控制，组件中使用位掩码来标识目标类型，包括 `GameComponent` （1 << 0）、`UI` （1 << 1）、和 `SmallMap` （1 << 2）。

所有实体的渲染由 `SRenderSystem` 统一处理，系统按渲染层级对实体进行排序后，遍历每个实体并根据其 `CRenderTargetComponent` 中的掩码信息决定其应当被绘制到哪些目标上。该系统支持多种类型的渲染组件，包括 `CSpriteRenderComponent` 用于静态精灵，`CAnimatedSpriteRenderComponent` 用于动画序列渲染，`CTextRenderComponent` 用于支持自动换行的文本渲染，以及 `CShapeRenderComponent` 用于绘制基本几何图形。所有这些组件都依赖于实体中是否具有 `CRenderComponent` （表示实体当前是否可见）、`CRenderLayerComponent` （决定绘制顺序）、`CRenderTargetComponent` （决定绘制目标）以及 `CGlobalTransform` （用于变换信息）的组合。

光照系统独立于主渲染流程，使用自定义片段着色器绘制光源。每个光源被表示为一个矩形形状，光照效果通过片段着色器中的径向衰减公式实现，并最终绘制到专门的 `illumination` 渲染目标中。

在主渲染阶段完成之后，整个渲染管线进入后处理阶段。后处理依赖多个着色器串联处理多个效果。首先应用像素化着色器，对图像进行采样降分辨率以模拟复古像素效果。接着应用 CRT 样式着色器，模拟老式显示器的视觉特征，如扫描线、色彩偏移（色差）以及桶形畸变。泛光处理包含两个阶段，首先提取图像中高亮区域，然后对提取结果进行高斯模糊处理，增强图像的光晕感。

在所有后处理效果完成之后，渲染管线进入最终合成阶段。此阶段将 `gameComponents` 渲染目标内容作为基础图层，叠加环境光照图层（使用乘法混合模式）以及动态光照图层（使用加法混合模式），之后再将 UI 图层绘制在最上层，完成最终画面输出。

实体的可见性由 `RenderUtils` 类提供接口进行集中控制。通过动态添加或移除 `CRenderComponent`，可控制某个实体及其全部子节点的渲染状态，这一机制适用于需要实现整体隐藏或显示的场景，如角色消失、UI 界面切换等功能。

AI 系统

本项目中的 AI 系统主要用于实现怪物的行为逻辑与子弹的自动追踪功能，整体采用有限状态机架构进行建模。各类行为状态与状态切换机制均通过 FSM 明确控制，从而实现不同游戏对象在不同情境下的反应逻辑。

怪物的 AI 状态机包括三个核心状态：**静止**、**索敌** 和 **死亡**。在静止状态下，怪物保持不动，处于待命状态。当进入索敌状态时，系统会尝试在怪物的感知范围内查找敌对目标（通常为玩家角色）。若成功找到目标，怪物将朝其方向移动，并发射一枚子弹。若在当前帧中未能发现敌人，则怪物会返回静止状态，继续等待目标进入视野。怪物在战斗过程中会响应玩家子弹的攻击，若被命中，其生命值将被扣除。生命值降低至 0 后，怪物会进入死亡状态，并在内部计划系统中注册销毁自身的操作，最终由引擎将其移出场景。

子弹的 AI 行为也具有一定智能，特别体现在玩家子弹的自动追踪能力上。玩家发射的子弹在创建后将尝试进行一次目标搜索操作，寻找当前场景中距离自身最近的敌对单位（即怪物）。一旦锁定目标，子弹会根据自身位置与目标位置之间的差向量计算出追踪方向。该差向量经过归一化后乘以一个预设的速度系数，形成子弹理想的目标速度。随后，子弹将当前速度与目标速度进行线性插值，从而平滑过渡到追踪状态，使其呈现出柔和的转向行为，实现更自然的自动追踪效果。

玩家本身的状态机则主要用于驱动动画系统。状态转换依据玩家的移动状态进行判定。当玩家开始移动时，动画状态从“待机”过渡到“移动”，并播放对应的行走动画；而当玩家停止移动时，动画状态则回到“待机”状态，展示静止贴图。此外，状态机还会检测玩家的水平方向速度，若速度为正，表示玩家朝右移动，贴图将维持默认朝向；若速度为负，则需将角色贴图进行水平翻转，以确保角色始终面向移动方向，增强画面表现力和交互反馈。